

RHCSA EX200 — Resumo Dia 03/05

LVM · VDO · Thin Provisioning · Swap · Partição Padrão

PROVA: 14/05

Hierarquia LVM — Memorize

PV — Physical Volume
tijolo (partição)

VG — Volume Group
pilha de tijolos

LV — Logical Volume
estrutura construída

⚠ Armadilhas Críticas

Certo	Errado
<code>-L</code> (maíusculo) = tamanho em MB/GB	<code>-l</code> (minúsculo) = número de extents
<code>lvextend -r</code> = expande LV + FS	sem <code>-r</code> = só o LV; o FS não cresce
Swap no fstab: <code>none</code> como mountpoint	usar <code>/swap</code> como mountpoint
Partição padrão: <code>mkfs</code> direto	usar <code>pvccreate</code> em partição padrão

BLOCO 1 — LVM Padrão

Fluxo completo (decore a sequência):

`fdisk → pvccreate → vgcreate → lvcreate → mkfs → fstab → lvextend -r`

```
# 1. Criar partição
fdisk /dev/sdb
    n → p (ou Enter em GPT) → Enter → Enter → +2G → t → 8e (Linux LVM) → w
partprobe /dev/sdb          # obrigatório

# 2. Physical Volume
pvccreate /dev/sdb1

# 3. Volume Group (PE padrão 4MB)
vgcreate myvg /dev/sdb1

# 4. Logical Volume
lvcreate -n mylv -L 500M myvg      # -L = tamanho
# lvcreate -n mylv -l 50 myvg      # -l = n° de extents

# 5. Formatar
mkfs.ext4 /dev/myvg/mylv

# 6. Montar persistente
mkdir /mylv
echo "/dev/myvg/mylv /mylv ext4 defaults 0 0" >> /etc/fstab
mount -a
df -h /mylv

# 7. Expandir LV + filesystem em UM comando
lvextend -r -L +500M /dev/myvg/mylv

# Verificação
pvs && vgs && lvs && lsblk
```

Exemplo completo (S1-Q6): disco `/dev/sdb`, VG `myvg`, LV `mylv` 500MiB → ext4 → `/mylv`, expandir +500MiB

```
fdisk /dev/sdb          # n → p → Enter → Enter → +1G → t → 8e → w
partprobe /dev/sdb
pvccreate /dev/sdb1
vgcreate myvg /dev/sdb1
lvcreate -n mylv -L 500M myvg
mkfs.ext4 /dev/myvg/mylv
mkdir /mylv
echo "/dev/myvg/mylv /mylv ext4 defaults 0 0" >> /etc/fstab
mount -a && df -h /mylv
lvextend -r -L +500M /dev/myvg/mylv
df -h /mylv
```

Crie a partição com `+1G` para ter espaço para os 500M iniciais + a expansão de +500M. `lvextend` sem `-r` expande apenas o container LV — o filesystem **não** cresce. `partprobe` é obrigatório após `fdisk`.

BLOCO 2 — PE Size Personalizado

O que é PE (Physical Extent)? É o bloco mínimo de alocação do LVM — como a unidade de cluster de um filesystem. Padrão: 4MB. Quando o exame pede PE de 16M, use `-s 16M` no `vgcreate`. LVs desse VG só podem ser criados em múltiplos de 16MB.

A conta que cai na prova:

- `50 extents × 16MB = 800MB` (tamanho resultante do LV)
- `800MB ÷ 16MB = 50 extents` (conta inversa)

Exemplo completo (S5-Q15): VG `myvg` com PE size 16MiB, LV `mylv` com 50 extents, xfs → `/mnt/mylv`

```
fdisk /dev/sdb      # n → Enter → Enter → +2G → t → 8e → w
partprobe /dev/sdb
pvcreate /dev/sdb1
vgcreate -s 16M myvg /dev/sdb1      # PE size personalizado
lvcreate -l 50 -n mylv myvg         # -l minúsculo = extents (50 × 16MB = 800MB)
mkfs.xfs /dev/myvg/mylv
mkdir /mnt/mylv
echo "/dev/myvg/mylv /mnt/mylv xfs defaults 0 0" >> /etc/fstab
mount -a
vgdisplay myvg | grep "PE Size"      # deve mostrar 16.00 MiB
lvdisplay /dev/myvg/mylv | grep "Current LE"  # deve mostrar 50
```

-l minúsculo = número de extents | -L maiúsculo = tamanho em MB/GB — não confundir!

BLOCO 3 — Partição Padrão (sem LVM)

Quando NÃO usar LVM: quando a questão pede "partição padrão", "formatar diretamente", ou não menciona PV/VG/LV. Fluxo: `fdisk → mkfs` (pula tudo do meio).

Por que UUID no fstab? Nomes de dispositivo (`/dev/sdb1`) podem mudar entre reboots. O UUID é permanente — identifica a partição pelo conteúdo, não pelo nome do dispositivo. Use `blkid` para obtê-lo.

Exemplo completo (S3-Q6): partição 512MiB em `/dev/sdb → ext4 → /mnt/data` via UUID, persistente

```
fdisk /dev/sdb      # n → Enter → Enter → +512M → w
partprobe /dev/sdb
mkfs.ext4 /dev/sdb1  # NÃO usar pvcreate aqui!
mkdir -p /mnt/data
blkid /dev/sdb1      # copiar o UUID mostrado
vim /etc/fstab
# UUID=<uuid-aqui> /mnt/data ext4 defaults 0 0
mount -a
df -h /mnt/data
```

Esta questão pede partição **padrão**. NÃO executar `pvcreate / vgcreate / lvcreate`.

BLOCO 4 — Swap

Três diferenças em relação a partições comuns:

- Tipo no fdisk: **82** (Linux Swap), não 8e (Linux LVM)
- Formatador: `mkswap`, não `mkfs`
- fstab: ponto de montagem = **none** | tipo = **swap**

Exemplo completo (S1-Q15): partição swap 512MiB

```
fdisk /dev/sdb
  n → Enter → Enter → +512M → t → 82 → w  # 82 = Linux Swap
partprobe /dev/sdb
mkswap /dev/sdb2      # ajustar número da partição
blkid /dev/sdb2       # copiar UUID
vim /etc/fstab
# UUID=<uuid> none swap defaults 0 0
#      ^^^^^ SEMPRE 'none' — nunca /swap ou outro caminho
swapon -a              # ativar sem reboot
free -h                # confirmar coluna Swap aumentou
```

Simulado	Tamanho	Partição típica
S1	512MiB	/dev/sdb2
S2	500MiB	/dev/sdb2
S5	1GiB	/dev/sdb2
S6	1GiB	/dev/sdb2

`swapon -a` ativa tudo do fstab com tipo swap — bom teste de que o fstab está correto. O número da partição depende de quantas já existem no disco.

BLOCO 5 — VDO & Thin Provisioning

VDO (Virtual Data Optimizer)

Tecnologia de deduplicação e compressão inline gerenciada pelo LVM.

- `--size` = espaço físico real consumido no VG
- `--virtualsize` = espaço apresentado ao SO (pode ser 4x-10x maior)
- RHEL 10 exige **mínimo ~5GB físico** para VDO (metadados do índice UDS)

Exemplo completo (S1-Q20): VG `myvg` já existente, VDO LV `vdo_lv`, 5GiB físico → 20GiB virtual, xfs → `/vdo_data`

```
dnf install lvm2 vdo -y
lvcreate --type vdo --name vdo_lv --size 5G --virtualsize 20G myvg
mkfs.xfs /dev/myvg/vdo_lv
mkdir /vdo_data
echo "/dev/myvg/vdo_lv /vdo_data xfs defaults 0 0" >> /etc/fstab
mount -a
vdostats --human-readable
lsblk
```

Thin Provisioning

Permite prometer mais espaço do que existe fisicamente. Dois objetos: **Thin Pool** (container físico, `-T` + `-L`) e **Thin Volume** (virtual, `-v` + `-T` referenciando o pool).

Exemplo completo (S3-Q27): VG `thin_vg`, Thin Pool `my_thin_pool` 500MiB, Thin Volume `thin_vol` 2GiB virtual, xfs → `/mnt/thin`

```
fdisk /dev/sdb          # criar partição 3, +1G, tipo 8e
partprobe /dev/sdb
pvcreate /dev/sdb3
vgcreate thin_vg /dev/sdb3

lvcreate -L 500M -T thin_vg/my_thin_pool          # cria o pool (500MB físico real)
lvcreate -V 2G -T thin_vg/my_thin_pool -n thin_vol # cria o volume (2GB virtual > pool!)

mkfs.xfs /dev/thin_vg/thin_vol
mkdir /mnt/thin
echo "/dev/thin_vg/thin_vol /mnt/thin xfs defaults 0 0" >> /etc/fstab
mount -a
lvs thin_vg      # mostra VSize (2G) e o pool (500M)
```

`-T` cria/referencia o thin pool. `-v` define o tamanho virtual. Sem `-T` na criação do pool: não é thin. Sem `-v` no volume: não é thin.

Técnica extra — `lvrename` (pode cair combinado com expansão do pool):

```
lvextend -L +500M vg1/thinpool1          # expandir o pool
lvrename vg1/thinpool1 vg1/thinpool2     # renomear pool
lvrename vg1/thinvol1 vg1/thinvol2       # renomear volume
```

Sintaxe: `lvrename VG/LV_ANTIGO VG/LV_NOVO` — o VG é obrigatório. **Não use** `mv` no `/dev` — isso só renomeia o symlink, não os metadados LVM internos.

📊 Tabela de Diferenças Críticas

Situação	Comando CORRETO	Erro comum
LV por tamanho	<code>lvcreate -L 500M myvg</code>	<code>lvcreate -l 500M</code>
LV por extents	<code>lvcreate -l 50 myvg</code>	<code>lvcreate -L 50</code>
Expandir + FS	<code>lvextend -r -L +500M ...</code>	esquecer o <code>-r</code>
PE personalizado	<code>vgcreate -s 16M vg /dev/sdb1</code>	criar VG sem <code>-s</code>
Swap no fstab	<code>UUID=xx none swap defaults 0 0</code>	<code>UUID=xx /swap swap ...</code>
Partição padrão	<code>mkfs.ext4 /dev/sdb1</code>	<code>pvcreate /dev/sdb1</code> (cria LVM)
VDO mínimo	<code>--size 5G</code>	<code>--size 1G</code> (falha por metadados)
Thin pool	<code>lvcreate -L 500M -T vg/pool</code>	esquecer o <code>-T</code>
Thin volume	<code>lvcreate -V 2G -T vg/pool -n vol</code>	esquecer o <code>-v</code>

✅ Checklist Final

LVM Padrão

- ☐ Fluxo completo (fdisk → pvs → vgs → lvs) sem consulta
- ☐ `lvextend -r` funciona — filesystem cresce junto com o LV
- ☐ fstab correto para LVM (`/dev/vg/lv`) + `mount -a` sem erro

PE Size Personalizado

- ☐ `vgcreate -s 16M` para PE personalizado
- ☐ `-l` (extents) vs `-L` (tamanho) — não confundir
- ☐ Sei fazer a conta: 50 extents × 16MB = 800MB

Partição Padrão

- ☐ `mkfs` direto na partição, SEM `pvcreate/vgcreate/lvcreate`
- ☐ UUID no fstab — obtido com `blkid`

Swap

- ☐ `fdisk` tipo 82 (swap, não 8e de LVM)
- ☐ fstab com `none` como mountpoint e `swap` como tipo
- ☐ `swapon -a` ativa | `free -h` confirma

VDO & Thin Provisioning

- ☐ VDO: `--size` (físico) vs `--virtualsize` (apresentado ao SO)
- ☐ VDO mínimo 5GB físico
- ☐ Thin pool: `lvcreate -L -T` | Thin volume: `lvcreate -V -T`
- ☐ `lvrename vg/old vg/new` funciona em pools e volumes